

Rapport d'avancement

Vital FOCHEUX

du 5 juin au 12 juin 2026

1 Notes de la dernière réunion

- Qu'il faut faire des schémas pour comprendre où sont situés les différents filtres dans le système
- Faire un filtre passe-haut pour avoir plus de modularité
- Faire en sorte que les filtres puissent être utilisés indépendamment des contrôleurs pour pouvoir les utiliser ailleurs
- Faire une figure pour représenter l'intégral au cours du temps pour les anti-windup
- Bien définir les équations mathématiques pour les filtres et les anti-windup

2 Depuis la dernière fois

2.1 Planning

Avec Anna on s'est fixé un planning pour les prochaines semaines jusqu'à la fin du mois de juin, qui est le suivant :

- du 8 au 12 juin :
 - Correction du modèle de drone → pour qu'il fonctionne avec une consigne plus cohérente (5 mètres) et qu'on observe des résultats différents entre avec/sans anti-windup
 - Séparation filtres/contrôleurs
 - Clarifier l'interface chips → montrer comment utiliser Filtre/Anti-windup en chips avec description bloc fonctionnel et documentation
 - Plan détaillé du rapport de stage
- du 15 au 19 juin :
 - Rédaction du package contrôleur chips (diagramme bloc fonctionnel + chips) → tester (via BIP)
 - Des plots et diagrammes (pour analyser les résultats)
 - Mise en commun avec Julien (travaux futurs)
- du 22 au 26 juin :
 - Rédaction du rapport de stage
 - Lien formel travaux Vital / Motif control

2.2 Filtres

Je me suis rendu compte que les exemples que j'utilisais sur les filtres utilisaient un contrôleur PID, mais qu'il n'y avait pas de boucle de rétroaction. J'ai sorti les filtres des contrôleurs pour les utiliser indépendamment comme ce qu'on aimerait pouvoir faire dans n'importe quel système.

2.2.1 Passe-Bas et Passe-Haut

Pour chacun de ces 2 filtres, je génère un ensemble de valeurs brut en utilisant cette équation :

$$y = \sin(1.2 * 2 * \pi * t) + 1.5 * \cos(0.8 * 2 * \pi * t) + 0.5 * \sin(2 * 2 * \pi * t)$$

Où t est la variable de temps allant de 0 à 150 secondes avec un pas de $\frac{1}{30}$ secondes.

Le but est de voir comment les filtres réagissent à différentes fréquences de signal.

Pour le filtre passe-bas, l'équation est la suivante :

$$\alpha = \tau / (\tau + dt)$$

$$y_n = \alpha * y_{n-1} + (1 - \alpha) * x_n$$

Où :

- τ est la constante de temps du filtre, qui détermine la rapidité avec laquelle le filtre réagit aux changements de signal
- dt est le pas de temps entre les échantillons
- y_n est la sortie du filtre à l'instant n
- y_{n-1} est la sortie du filtre à l'instant $n-1$
- x_n est l'entrée du filtre à l'instant n

Pour le filtre passe-haut, l'équation est la suivante :

$$\alpha = \tau / (\tau + dt)$$

$$y_n = \alpha * (y_{n-1} + x_n - x_{n-1})$$

Où :

- τ est la constante de temps du filtre, qui détermine la rapidité avec laquelle le filtre réagit aux changements de signal
- dt est le pas de temps entre les échantillons
- y_n est la sortie du filtre à l'instant n
- y_{n-1} est la sortie du filtre à l'instant $n-1$
- x_n est l'entrée du filtre à l'instant n
- x_{n-1} est l'entrée du filtre à l'instant $n-1$

Pour les 2 filtres, j'ai choisi une constante de temps τ égale à $\frac{1}{(2 * \pi * 3.667)} \approx 0.043$ secondes et un pas de temps dt de $\frac{1}{30}$ secondes.

On peut voir sur les schémas à quel niveau du système les filtres sont appliqués. (cf figures 4, 5 et 6)

Le filtre passe-bas laisse passer les basses fréquences et atténue les hautes fréquences, tandis que le filtre passe-haut fait l'inverse. En combinant les deux filtres, on peut créer un filtre passe-bande qui laisse passer une plage de fréquences spécifique.

Pour le filtre passe-bas (cf figure 1), on peut voir que les hautes fréquences sont atténuées, tandis que les basses fréquences sont préservées. Pour le filtre passe-haut (cf figure 3), on peut voir que les basses fréquences sont atténuées, tandis que les hautes fréquences sont préservées. Enfin, pour le filtre passe-haut puis passe-bas (cf figure 2), on peut voir que les fréquences ne dépassent pas une certaine plage, ce qui correspond à un filtre passe-bande.

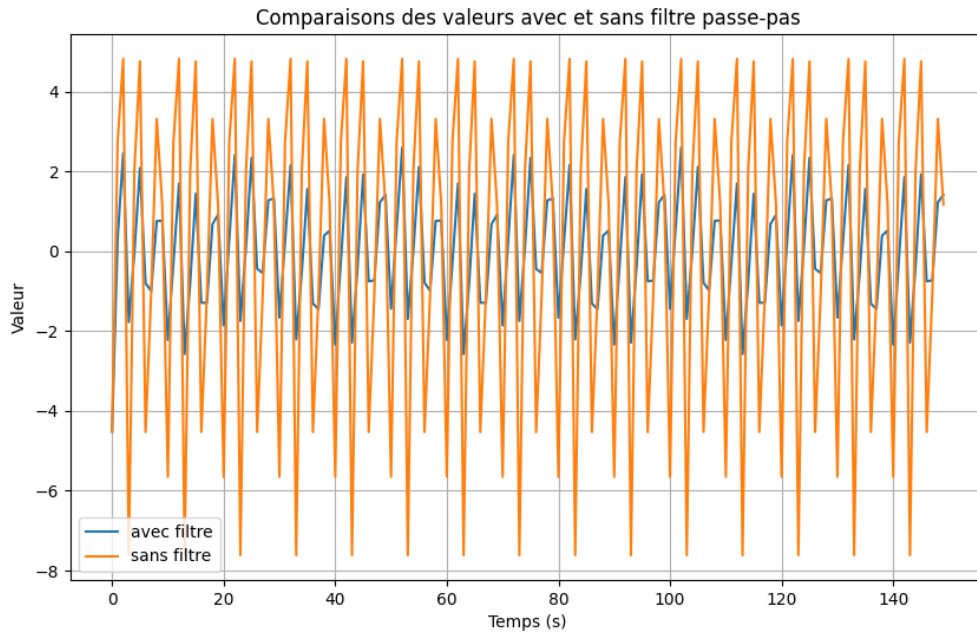


Figure 1: Résultat du filtre passe-bas sur les données

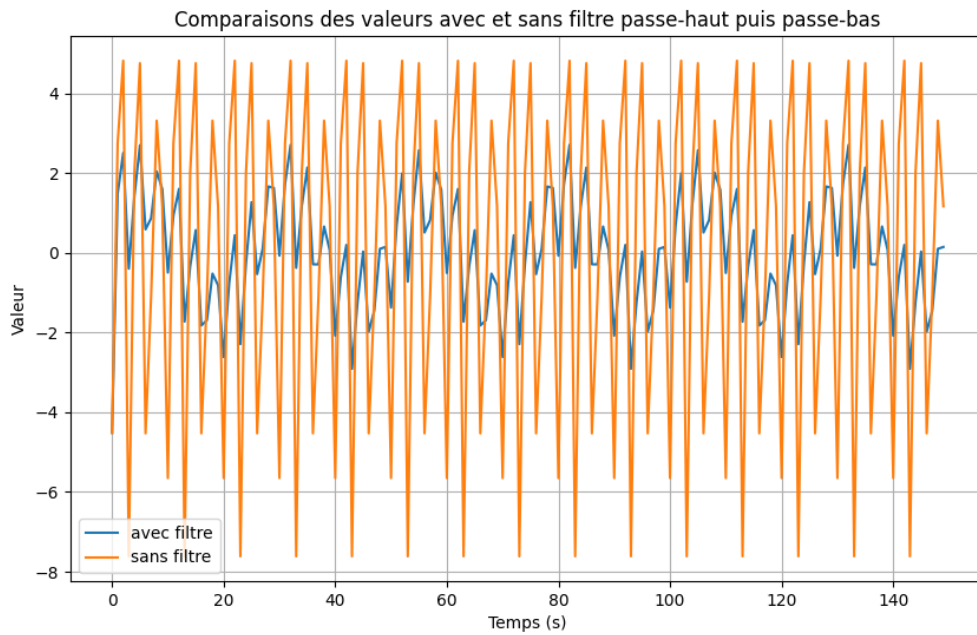


Figure 2: Résultat du filtre passe-haut puis passe-bas sur les données

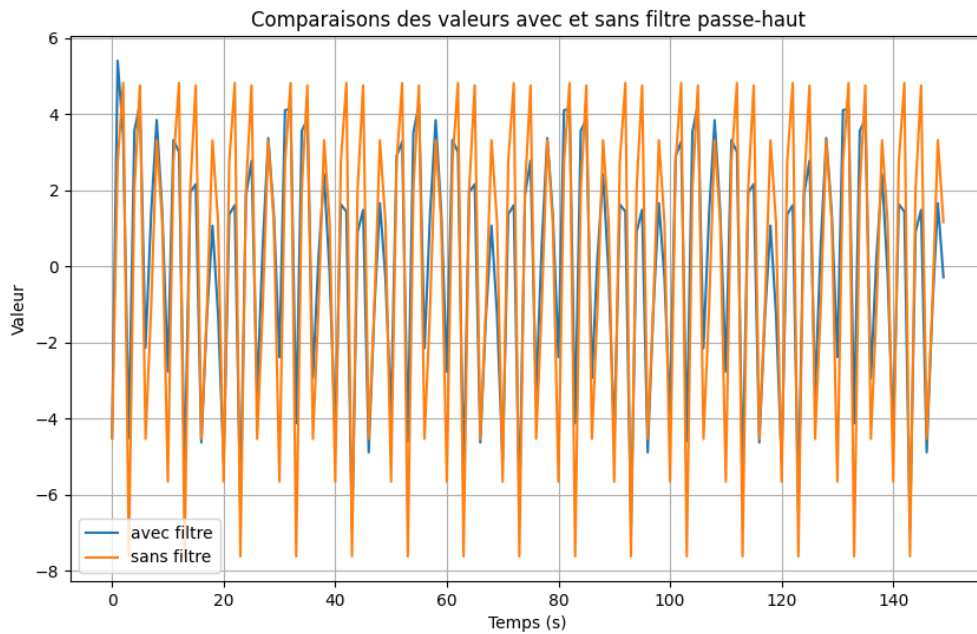


Figure 3: Résultat du filtre passe-haut sur les données

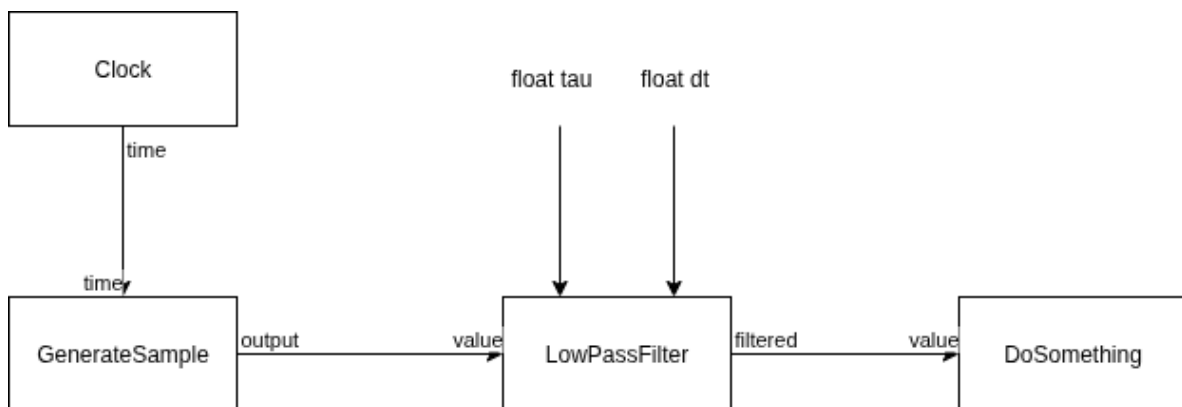


Figure 4: Schéma du filtre passe-bas

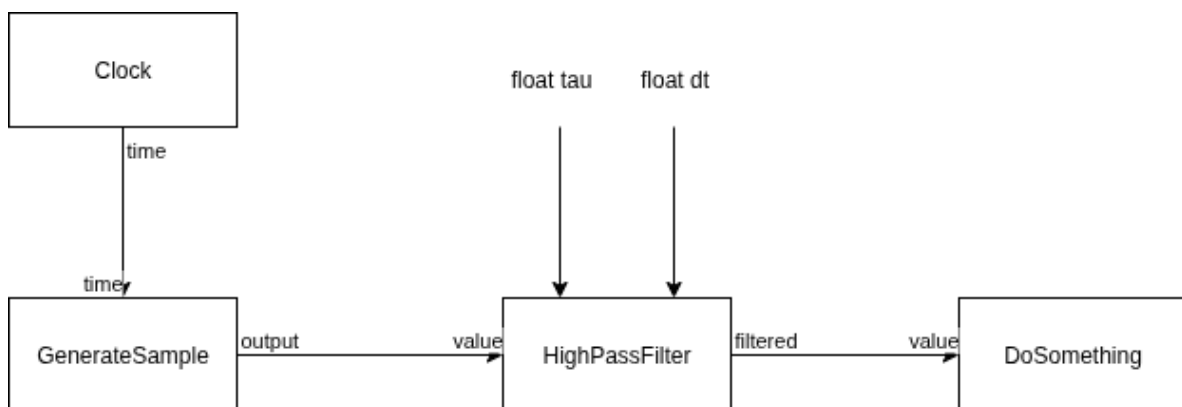


Figure 5: Schéma du filtre passe-haut

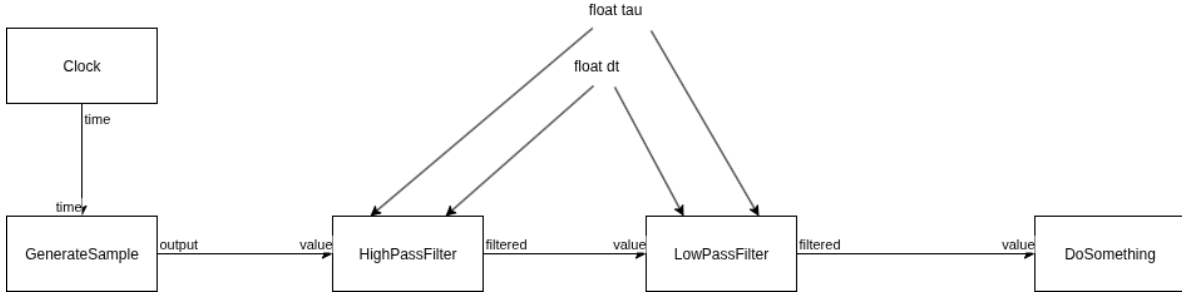


Figure 6: Schéma du filtre passe-haut puis passe-bas

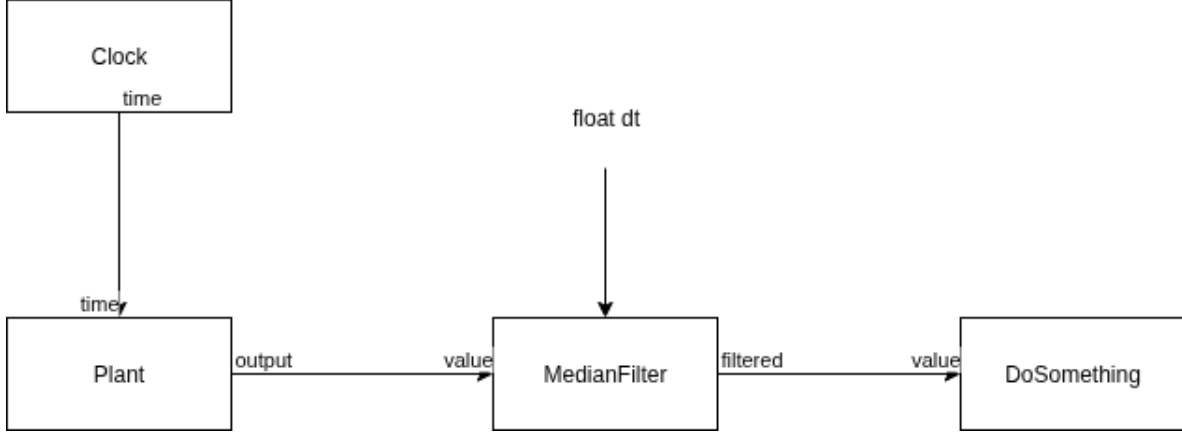


Figure 7: Schéma du filtre médian

2.2.2 Médian

Tout comme les exemples précédents, j'utilise une équation pour générer un ensemble de valeurs brut :

$$y = \sin(2 * \pi * freq * time)$$

où :

- freq est la fréquence du signal, que j'ai choisie égale à 0.2 Hz
- time est la variable de temps allant de 0 à 200 secondes avec un pas de 0.05 secondes

J'ajoute ensuite du bruit aléatoire à ces données pour simuler des données réelles, ce qui me permet de tester l'efficacité du filtre médian pour réduire le bruit. J'ai choisi d'ajouter des pics tous les 40 secondes pour simuler une valeur aberrante dans les données, ce qui est un cas courant dans les données réelles.

Le filtre médian est un filtre non linéaire qui remplace chaque échantillon de signal par la médiane des échantillons voisins. Ici les échantillons voisins sont les 5 échantillons précédents (ou suivants en fonction de la position de l'échantillon).

2.3 Anti-windup

Maintenant j'utilise le même exemple (celui du drone) avec 2 anti-windup différents : le clamp et le back-calculation. Pour permettre de comparer les 2 méthodes, j'ai utilisé les mêmes données pour les 2 méthodes.

Pour calculer l'altitude du drone en fonction du temps, j'utilise les équations suivantes :

$$K = 4 * k_p * \left(\frac{2\pi}{60}\right)^2$$

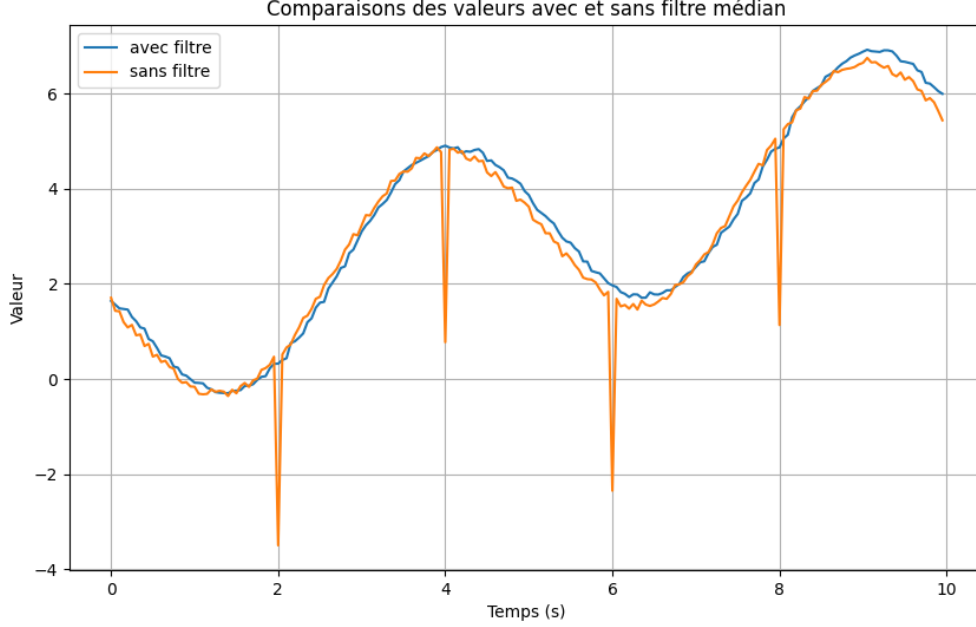


Figure 8: Résultat du filtre médian sur les données

$$F_p = K * \Omega_{rpm}^2$$

$$drag = DRAG_COEFF * velocity$$

$$\alpha = \frac{F_p}{m} - g - \left(\frac{drag}{m}\right)$$

Où :

- k_p est la constante de poussée propre à chaque hélice, ici égale à 0.29
- Ω_{rpm} est la vitesse de rotation de l'hélice en tours par minute, qui correspond à la commande du contrôleur
- $DRAG_COEFF$ est le coefficient de traînée, que j'ai choisi égal à 0.8
- $velocity$ est la vitesse du drone, que je calcule en intégrant l'accélération α au cours du temps
- m est la masse du drone, que j'ai choisie égale à 0.2 kg
- g est l'accélération due à la gravité, que j'ai choisie égale à 9.81 m/s²

L'objectif est de faire en sorte que le drone atteigne une altitude de 5 mètres et qu'on le garde au sol pendant 10 secondes et de voir comment les différentes méthodes d'anti-windup affectent la performance du système.

2.3.1 Clamp

L'anti-windup clamp consiste à limiter la valeur de l'intégral pour éviter qu'elle ne dépasse 2 certaines limites (maximal et minimal). Si l'intégral dépasse ces limites, il est "clampé" à la limite correspondante. Ici la limite maximale est de 30 et la limite minimale est de -30.

L'observation flagrante est qu'au niveau de l'intégral (cf figure 10) on voit que l'intégral est clampé à 30 pendant une partie du temps, ce qui correspond à la période où le drone est au sol et que la commande est maximale pour essayer de faire décoller le drone.

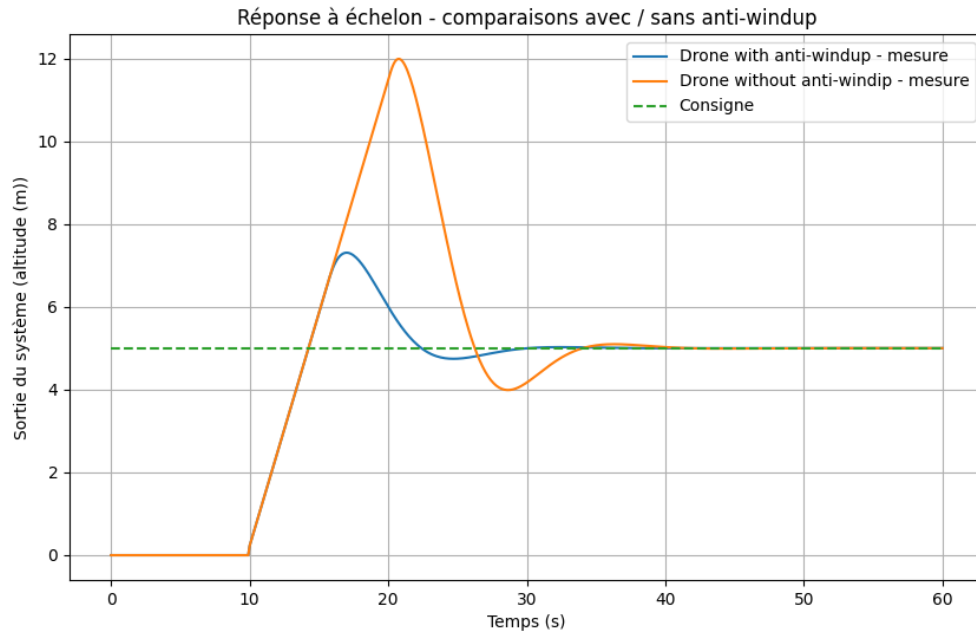


Figure 9: Résultat de l'anti-windup sur les données

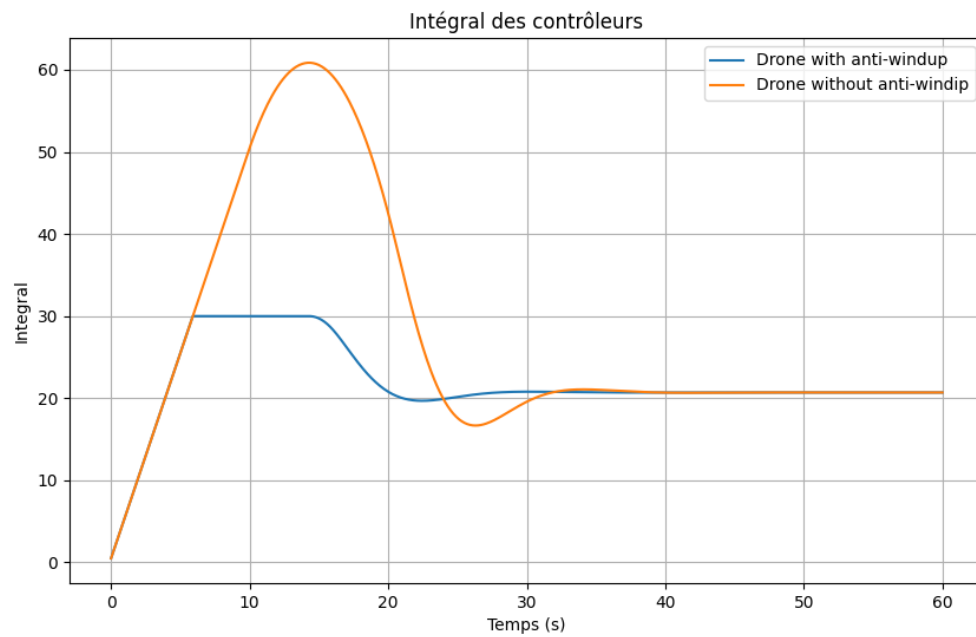


Figure 10: Intégral au cours du temps avec anti-windup

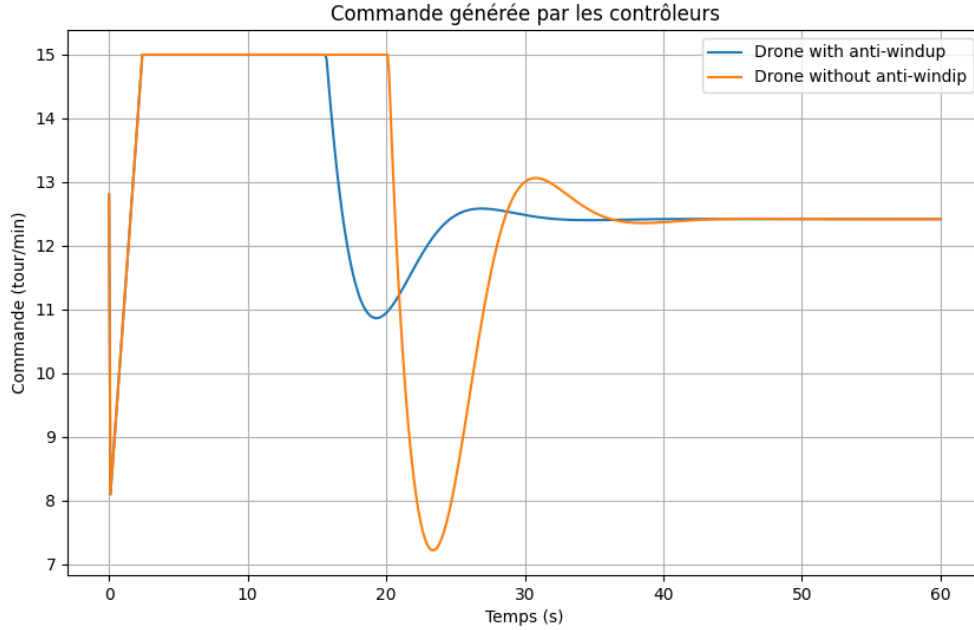


Figure 11: Résultat de l'anti-windup sur les données

2.3.2 Back-Calculation

L'anti-windup back-calculation consiste à calculer une correction à appliquer à l'intégral en fonction de la différence entre la commande maximale et la commande réelle. Cette correction est ensuite soustraite de l'intégral pour éviter qu'il ne dépasse les limites. Ici la correction est calculée en utilisant une constante de gain de back-calculation égale à 0.2. La correction est calculée comme suit :

$$integral = integral + \frac{(satOutput - rawOutput)}{m_Tt * dt}$$

où :

- *satOutput* est la commande maximale (ou minimale) à laquelle la sortie est limitée
- *rawOutput* est la commande réelle calculée par le contrôleur
- *m.Tt* est la constante de gain de back-calculation, que j'ai choisie égale à 0.2
- *dt* est le pas de temps entre les échantillons, que j'ai choisi égal à 0.1 secondes

On peut se rendre compte que l'anti-windup back-calculation permet de réduire l'intégral de manière plus progressive que l'anti-windup clamp, ce qui peut permettre d'améliorer la performance du système en évitant les changements brusques de commande. (cf figure 13 et 10) De même pour la commande de l'hélice, on peut voir que l'anti-windup back-calculation permet d'avoir une commande plus fluide que l'anti-windup clamp, qui peut avoir des changements brusques de commande lorsque l'intégral est clamped. (cf figure 14 et 11) Et pour la montée en altitude, on peut voir que les 2 méthodes permettent au drone d'atteindre l'altitude cible de 5 mètres, mais que l'anti-windup back-calculation permet une montée plus fluide et moins brusque que l'anti-windup clamp. (cf figure 12 et 9)

3 Clarification interface chips

J'ai clarifié l'interface chips pour les filtres et les anti-windup en créant une documentation détaillée qui explique comment utiliser ces outils dans différents systèmes. J'ai également créé des diagrammes de blocs fonctionnels pour montrer comment les filtres et les anti-windup peuvent être intégrés dans différents systèmes de contrôle, ce qui peut aider à comprendre comment les utiliser de manière efficace.

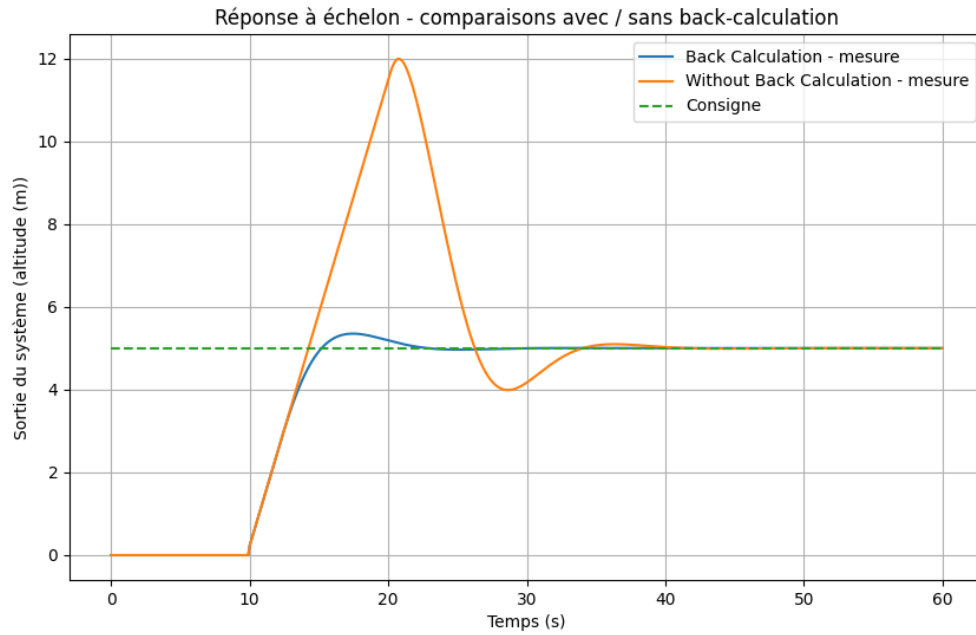


Figure 12: Résultat de l'anti-windup sur les données

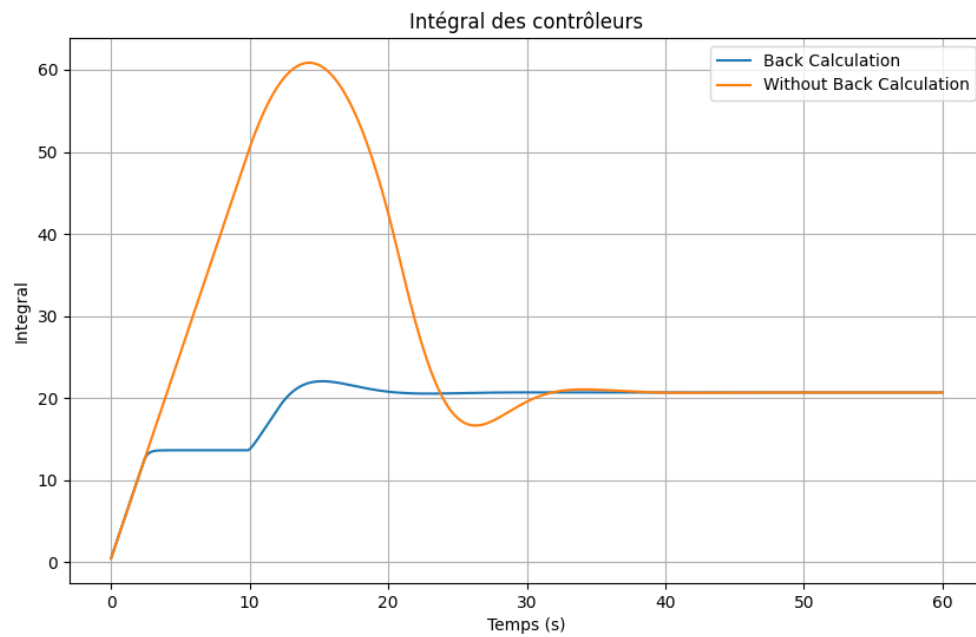


Figure 13: Intégral au cours du temps avec anti-windup

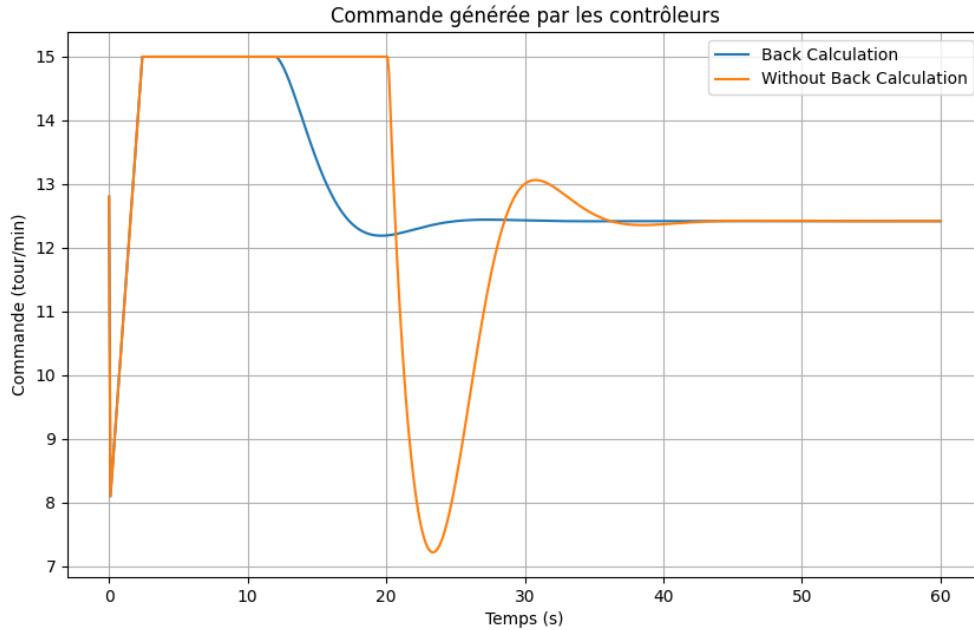


Figure 14: Résultat de l'anti-windup sur les données

3.1 Filtres

Ici, on va montrer avec un filtre passe-bas mais cela revient à la même chose pour les autres filtres.

Le diagramme de bloc fonctionnel pour le filtre passe-bas est le suivant (cf figure 4) et le programme chips correspondant est le suivant :

```
logical Clock() init {
  float dt = 0;
}then{
  dt = dt + 0.1;
} -> time(dt);

logical GenerateSample(float time)
-> output(// Some value )

logical DoSomething(float value) then{
  // Do something
}

system{
  LowPassFilter lp;
  GenerateSample sample;
  Clock clock;
  DoSomething dostg;
  float dt = 0.1;
  float tau = // Some value
  sample.time(clock.time);
  lp{dt: dt, tau: tau, value: sample.output};
  dostg.value(lp.filtered);
}
```

On pourrait lui associer une documentation qui explique comment utiliser le filtre (comme man

pour les commandes bash) et qui ressemblerait à cela :

LowPassFilter - A low-pass filter that allows low frequencies to pass through while
↪ attenuating high frequencies.

Usage:

```
LowPassFilter{dt: <float>, tau: <float>, value: <float>}
```

Parameters:

dt: The time step between samples, which determines how often the filter is
↪ updated.

tau: The time constant of the filter, which determines how quickly the filter
↪ responds to changes in the input signal.

value: The input signal to be filtered, which is a float value that can be
↪ generated by any logical block in the system.

Output:

filtered: The filtered output signal, which is a float value that can be used by
↪ any logical block in the system.

Description:

The LowPassFilter takes an input signal and applies a low-pass filter to it,
↪ allowing low frequencies to pass
through while attenuating high frequencies. The filter is updated at each time
↪ step defined by the dt parameter,
and the response of the filter is determined by the tau parameter. The filtered
↪ output can be used in any part
of the system, making it a versatile tool for signal processing in various
↪ applications.

3.2 Anti-windup

Pour l'anti-windup, on pourrait faire la même chose que pour les filtres en créant un diagramme de bloc fonctionnel pour chaque méthode d'anti-windup et en créant une documentation détaillée pour expliquer comment les utiliser dans différents systèmes de contrôle.

Le problème que je rencontre pour faire cela, c'est que l'anti-windup est directement intégré dans le contrôleur, ce qui rend difficile de l'utiliser de manière indépendante dans différents systèmes. Car chaque anti-windup a des paramètres spécifiques qui permettent de l'adapter à différents systèmes de contrôle. La figure 15 montre une idée qui sera expliquée plus en détails lors de la réunion.

4 Plan rapport

Pour le rapport de stage, je vais suivre le plan suivant :

1. Introduction
2. Présentation FEMTO-ST
3. Littérature, contexte et objectifs → pour expliquer les différentes lectures que j'ai pu faire durant le stage et comment elles m'ont permis de mieux comprendre les différents concepts et les mettre en lien avec les travaux que j'ai pu faire. Puis expliquer les objectifs du stage.
4. Parseur/XMI → expliquer comment a été finalisé le parseur et comment il fonctionne, avec des exemples d'utilisation. De même pour la sérialisation vers XMI.
5. Outils de la théorie de contrôle → Expliquer les différents outils qui ont pu être développés (filtres/anti-windups) et comment ils fonctionnent, avec des exemples d'utilisation. Expliquer comment ils peuvent être utilisés dans différents systèmes de contrôle.
6. Conclusion et perspectives

Le design pattern et les motifs de contrôle ne sont pas dans le plan car ils ne sont pas encore finalisés ou commencés, mais ils pourraient être ajoutés dans la section "Outils de la théorie de contrôle" ou dans une section séparée si nécessaire.

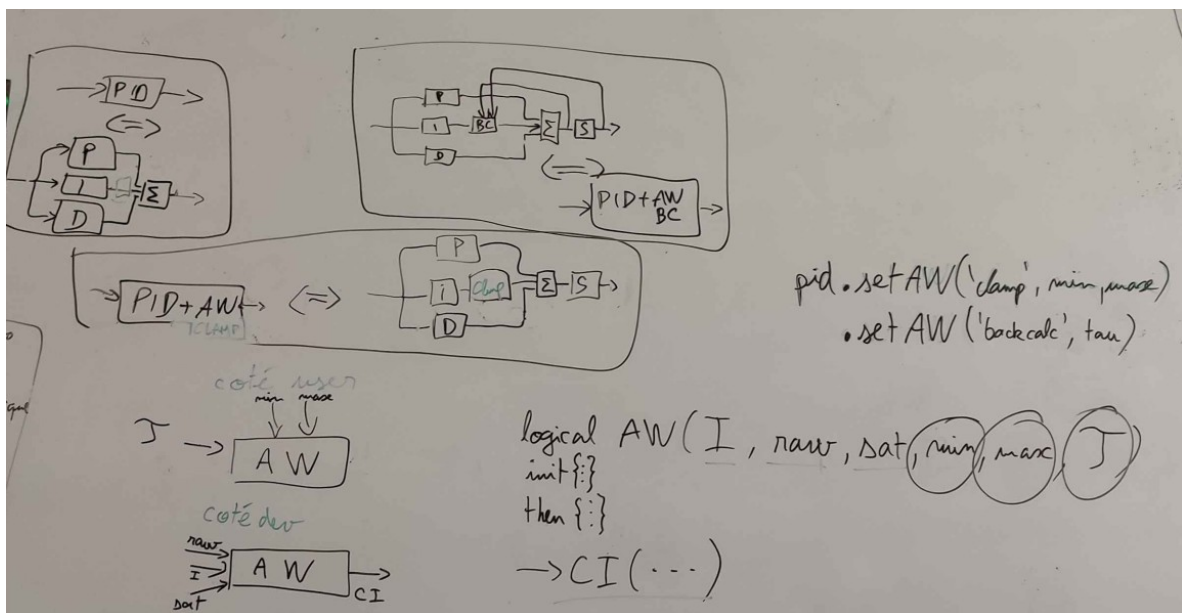


Figure 15: Idée pour l'antiwindup en CHIPS